

Αλγόριθμοι και Πολυπλοκότητα

ΦΡΟΝΤΙΣΤΗΡΙΑΚΕΣ ΑΣΚΗΣΕΙΣ (11) [τελευταίο]

Επανάληψη

11/6/2024

Έφη Μαλέσιου
Διδακτορική φοιτήτρια

ΑΣΚΗΣΗ 1

- Είναι πολυωνυμικά φραγμένη η συνάρτηση;
 - $\lceil \log(n) \rceil!$
 - $\lceil \log(\log(n)) \rceil!$

Για να αποδείξουμε ότι μία συνάρτηση $f(n)$ είναι πολυωνυμικά φραγμένη, αρκεί να αποδείξουμε ότι $\log(f(n)) = O(\log(n))$:

- αν η $f(n)$ είναι πολυωνυμικά φραγμένη, υπάρχουν σταθερές c, k, n_0 τ.ώ. $f(n) \leq c * n^k$.
Συνεπώς, $\log(f(n)) \leq c * \log(n^k) = c * k * \log(n) \Rightarrow \log(f(n)) = O(\log(n))$
- αν ισχύει $\log(f(n)) = O(\log(n))$, τότε η $f(n)$ είναι πολυωνυμικά φραγμένη

Επίσης, γνωρίζουμε ότι:

- $\log(n!) = \Theta(n \log(n))$
- $\lceil \log(n) \rceil < \log(n) + 1 \leq 2 \log(n) = \Theta(\log(n)), n \geq 2$
- $\log^a(n) = (\log(n))^a = o(n^b)$, σταθερές $a, b > 0$

Οπότε:

- $\log(\lceil \log(n) \rceil!) = \Theta(\lceil \log(n) \rceil \log(\lceil \log(n) \rceil))$
 $= \Theta(\log(n) \log(\log(n)))$
 $= \omega(\log(n)) \Rightarrow$ δεν είναι $O(\log(n))$
Άρα, η $\lceil \log(n) \rceil!$ δεν είναι πολυωνυμικά φραγμένη.
- $\log(\lceil \log(\log(n)) \rceil!) = \Theta(\lceil \log(\log(n)) \rceil \log(\lceil \log(\log(n)) \rceil))$
 $= \Theta(\log(\log(n)) \log(\log(\log(n))))$
 $= o(\log(\log(n))^2)$
 $= o(\log(n))$
 $= O(\log(n)) \Rightarrow$ είναι $O(\log(n))$
Άρα, η $\lceil \log(\log(n)) \rceil!$ είναι πολυωνυμικά φραγμένη.

ΑΣΚΗΣΗ 2

- Πολυπλοκότητα χρόνου αλγορίθμου:

Algorithm 3

```
1:  $var \leftarrow 0$ 
2: for  $i \leftarrow 0$  to  $n - 2$  with step 1 do
3:   for  $j \leftarrow i$  to  $n$  with step 1 do
4:     for  $k \leftarrow 0$  to  $2\sqrt{n}$  with step 1 do
5:        $var \leftarrow i + j + k$ 
6:     end for
7:   end for
8: end for
```

1^η προσέγγιση:

- 1^ο for (εξωτερικότερο): $(n-2 - 0 + 1) = n-1$ επαναλήψεις, $O(n)$
- 2^ο for: $i, (i+1), (i+2), \dots, (n-3), (n-2)$ επαναλήψεις, $O(n)$
- 3^ο for: $(2n^{0.5} - 0 + 1)$ επαναλήψεις, $O(n^{0.5})$
- Υπολογισμός, $var = i + j + k$: $O(1)$
- Συνολικά: $O(n) * O(n) * O(n^{0.5}) * O(1) = O(n^{2.5})$

2^η προσέγγιση:

$$\begin{aligned} & \sum_{i=0}^{n-2} \left(\sum_{j=i}^n \left(\sum_{k=0}^{2n^{0.5}} O(1) \right) \right) = \\ & \sum_{i=0}^{n-2} \left(\sum_{j=i}^n (2n^{0.5} - 0 + 1) c \right) = \\ & \sum_{i=0}^{n-2} \left((2n^{0.5} + 1) \sum_{j=i}^n c \right) = \\ & \sum_{i=0}^{n-2} \left((2n^{0.5} + 1)(n - i + 1) c \right) = \\ & \sum_{i=0}^{n-2} (2n^{1.5} + 2n^{0.5}i + 2n^{0.5} + n - i + 1) c = \\ & c (2n^{1.5} + 2n^{0.5} + n + 1) \sum_{i=0}^{n-2} 1 - c (1 + 2n^{0.5}) \sum_{i=0}^{n-2} i = \\ & c (2n^{1.5} + 2n^{0.5} + n + 1)(n - 2 - 0 + 1) - c (1 + 2n^{0.5})(0.5(n - 2)(n - 1)) = \\ & c(n^{2.5} + 0.5n^2 + 3n^{1.5} + 1.5n - 4n^{0.5} - 2) = \\ & \Theta(n^{2.5}) \end{aligned}$$

(B) Ένα πρόβλημα Π επιλύεται με τους παρακάτω δύο αναδρομικούς αλγόριθμους για στιγμιότυπα μεγέθους n :

i. Ο αλγόριθμος A1 διασπά το πρόβλημα σε 9 υποπροβλήματα μεγέθους $n/3$, τα επιλύει και στη συνέχεια συνθέτει τις λύσεις τους σε χρόνο n .

ii. Ο αλγόριθμος A2 διασπά το πρόβλημα σε 2 υποπροβλήματα μεγέθους $n/2$, τα επιλύει και στη συνέχεια συνθέτει τις λύσεις τους σε χρόνο cn για κάποια σταθερά c .

Να γράψετε τις αναδρομικές εξισώσεις που δίνουν τον χρόνο εκτέλεσης των αλγορίθμων A1 και A2 και στη συνέχεια να τις επιλύσετε με χρήση του Θεωρήματος Κυριαρχίας.

$$T_1(n) = 9T_1\left(\frac{n}{3}\right) + n$$

$$a = 9 \geq 1, b = 3 > 1, f(n) = n > 0$$

$$n^{\log_b a} = n^{\log_3 9} = n^2$$

$$f(n) = n = O(n^{2-0.5}), \varepsilon = 0.5 > 0$$

$$\Pi 1: T_1(n) = \Theta(n^2)$$

$$T_2(n) = 2T_2\left(\frac{n}{2}\right) + \Theta(n), cn = \Theta(n)$$

$$a = 2 \geq 1, b = 2 > 1, f(n) = cn > 0, c > 0$$

$$n^{\log_b a} = n^{\log_2 2} = n$$

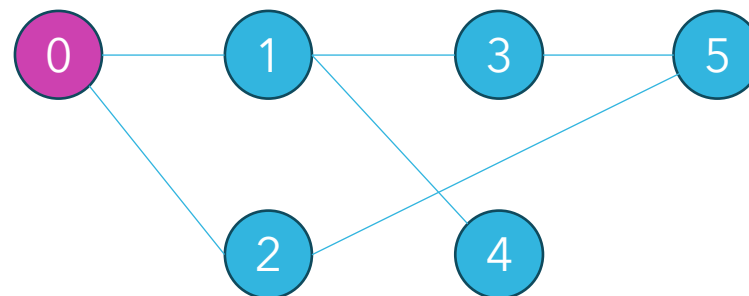
$$f(n) = cn = \Theta(n^{\log_b a}) = \Theta(n)$$

$$\Pi 2: T_2(n) = \Theta(n \log n)$$

Δίνεται γράφος $G = (V, E, W)$ με $|V|$ κόμβους, $|E|$ ακμές και ένα ίδιο μη αρνητικό βάρος $W(e) = C$ σε κάθε ακμή e . Η αναπαράσταση του γράφου είναι με λίστες γειτνίασης.

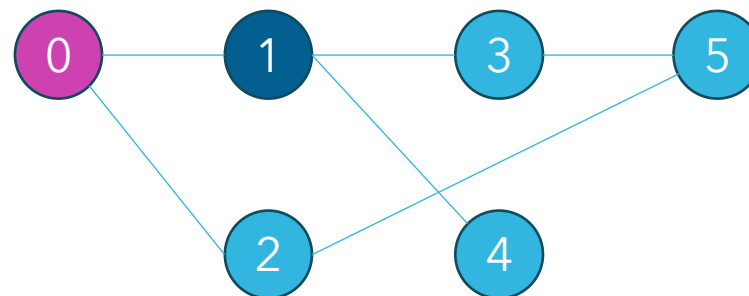
i. Να δοθεί αλγόριθμος σε φυσική γλώσσα, βέλτιστης πολυπλοκότητας, που βρίσκει το κόστος του συντομότερου μονοπατιού από δεδομένο κόμβο s προς κάθε άλλο κόμβο.

- Επειδή το βάρος είναι μη-αρνητικό και κοινό σε όλες τις ακμές, σημασία έχει ο αριθμός των ακμών που συνιστούν το μονοπάτι, άρα μπορούμε να εκτελέσουμε τον αλγόριθμο BFS, με πολυπλοκότητα $O(|V| + |E|)$.
 - δίνουμε αριθμητική ετικέτα σε κάθε κόμβο ($s \Rightarrow 0, 1, \dots, |V|$)
 - σε έναν πίνακα (dist), μήκους $|V|$, που αρχικοποιείται με τιμές, 0 για τον s και $+\infty$ για τους υπόλοιπους, αποθηκεύουμε το κόστος του συντομότερου μονοπατιού για εκάστοτε κόμβο (από τον s προς κάθε άλλο)
 - αρχικοποιούμε μια ουρά (queue), στην οποία τοποθετούμε τον κόμβο s
 - σε έναν πίνακα (visited), μήκους $|V|$, που αρχικοποιείται με τιμές 0 (εκτός του s , που είναι 1), αποθηκεύουμε την πληροφορία επίσκεψης της κάθε κορυφής (0 ή 1)
 - Ενόσω η ουρά δεν είναι κενή:
 - αφαιρούμε έναν κόμβο x από την ουρά
 - για κάθε γειτονικό του κόμβο, u , αν δεν έχουμε επισκεφθεί τον u
 - $\text{dist}[u] = \text{dist}[x] + 1$ (γιατί $\text{dist}[u] > \text{dist}[x] + 1$)
 - τοποθέτησε τον u στην ουρά
 - $\text{visited}[u] = 1$



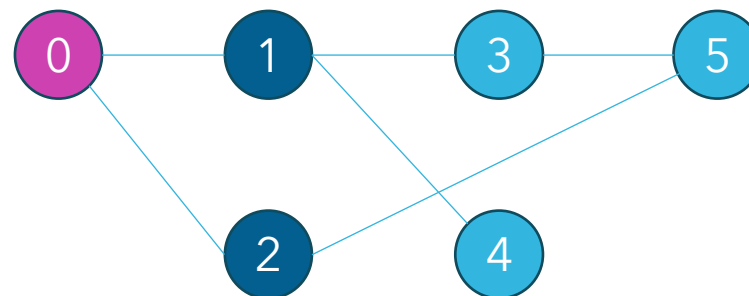
	0	1	2	3	4	5
dist	0	∞	∞	∞	∞	∞
visited	1	0	0	0	0	0
queue	0					

- Επειδή το βάρος είναι μη-αρνητικό και κοινό σε όλες τις ακμές, σημασία έχει ο αριθμός των ακμών που συνιστούν το μονοπάτι, άρα μπορούμε να εκτελέσουμε τον αλγόριθμο BFS, με πολυπλοκότητα $O(|V| + |E|)$.
 - δίνουμε αριθμητική ετικέτα σε κάθε κόμβο ($s \Rightarrow 0, 1, \dots, |V|$)
 - σε έναν πίνακα (dist), μήκους $|V|$, που αρχικοποιείται με τιμές, 0 για τον s και $+\infty$ για τους υπόλοιπους, αποθηκεύουμε το κόστος του συντομότερου μονοπατιού για εκάστοτε κόμβο (από τον s προς κάθε άλλο)
 - αρχικοποιούμε μια ουρά (queue), στην οποία τοποθετούμε τον κόμβο s
 - σε έναν πίνακα (visited), μήκους $|V|$, που αρχικοποιείται με τιμές 0 (εκτός του s , που είναι 1), αποθηκεύουμε την πληροφορία επίσκεψης της κάθε κορυφής (0 ή 1)
 - Ενόσω η ουρά δεν είναι κενή:
 - αφαιρούμε έναν κόμβο x από την ουρά
 - για κάθε γειτονικό του κόμβο, u , αν δεν έχουμε επισκεφθεί τον u
 - $\text{dist}[u] = \text{dist}[x] + 1$ (γιατί $\text{dist}[u] > \text{dist}[x] + 1$)
 - τοποθέτησε τον u στην ουρά
 - $\text{visited}[u] = 1$



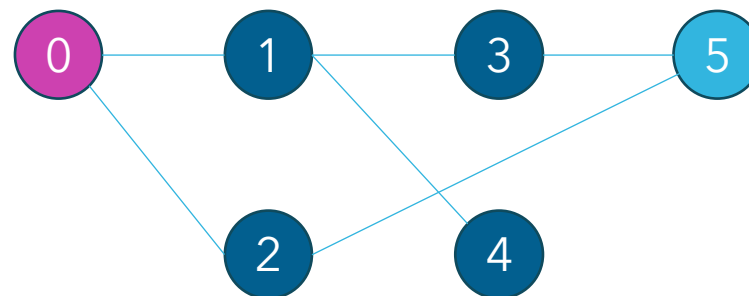
	0	1	2	3	4	5
dist	0	1	∞	∞	∞	∞
visited	1	1	0	0	0	0
queue	1					

- Επειδή το βάρος είναι μη-αρνητικό και κοινό σε όλες τις ακμές, σημασία έχει ο αριθμός των ακμών που συνιστούν το μονοπάτι, άρα μπορούμε να εκτελέσουμε τον αλγόριθμο BFS, με πολυπλοκότητα $O(|V| + |E|)$.
 - δίνουμε αριθμητική ετικέτα σε κάθε κόμβο ($s \Rightarrow 0, 1, \dots, |V|$)
 - σε έναν πίνακα (`dist`), μήκους $|V|$, που αρχικοποιείται με τιμές, 0 για τον s και $+\infty$ για τους υπόλοιπους, αποθηκεύουμε το κόστος του συντομότερου μονοπατιού για εκάστοτε κόμβο (από τον s προς κάθε άλλο)
 - αρχικοποιούμε μια ουρά (`queue`), στην οποία τοποθετούμε τον κόμβο s
 - σε έναν πίνακα (`visited`), μήκους $|V|$, που αρχικοποιείται με τιμές 0 (εκτός του s , που είναι 1), αποθηκεύουμε την πληροφορία επίσκεψης της κάθε κορυφής (0 ή 1)
 - Ενόσω η ουρά δεν είναι κενή:
 - αφαιρούμε έναν κόμβο x από την ουρά
 - για κάθε γειτονικό του κόμβο, u , αν δεν έχουμε επισκεφθεί τον u
 - $\text{dist}[u] = \text{dist}[x] + 1$ (γιατί $\text{dist}[u] > \text{dist}[x] + 1$)
 - τοποθέτησε τον u στην ουρά
 - $\text{visited}[u] = 1$



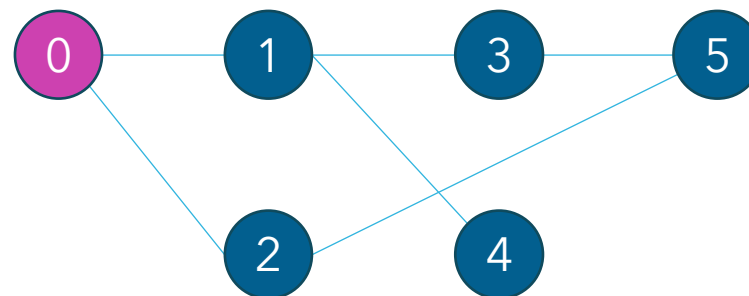
	0	1	2	3	4	5
dist	0	1	1	∞	∞	∞
visited	1	1	1	0	0	0
queue	1	2				

- Επειδή το βάρος είναι μη-αρνητικό και κοινό σε όλες τις ακμές, σημασία έχει ο αριθμός των ακμών που συνιστούν το μονοπάτι, άρα μπορούμε να εκτελέσουμε τον αλγόριθμο BFS, με πολυπλοκότητα $O(|V| + |E|)$.
 - δίνουμε αριθμητική ετικέτα σε κάθε κόμβο ($s \Rightarrow 0, 1, \dots, |V|$)
 - σε έναν πίνακα (dist), μήκους $|V|$, που αρχικοποιείται με τιμές, 0 για τον s και +άπειρο για τους υπόλοιπους, αποθηκεύουμε το κόστος του συντομότερου μονοπατιού για εκάστοτε κόμβο (από τον s προς κάθε άλλο)
 - αρχικοποιούμε μια ουρά (queue), στην οποία τοποθετούμε τον κόμβο s
 - σε έναν πίνακα (visited), μήκους $|V|$, που αρχικοποιείται με τιμές 0 (εκτός του s, που είναι 1), αποθηκεύουμε την πληροφορία επίσκεψης της κάθε κορυφής (0 ή 1)
 - Ενόσω η ουρά δεν είναι κενή:
 - αφαιρούμε έναν κόμβο x από την ουρά
 - για κάθε γειτονικό του κόμβο, u, αν δεν έχουμε επισκεφθεί τον u
 - $\text{dist}[u] = \text{dist}[x] + 1$ (γιατί $\text{dist}[u] > \text{dist}[x] + 1$)
 - τοποθέτησε τον u στην ουρά
 - $\text{visited}[u] = 1$



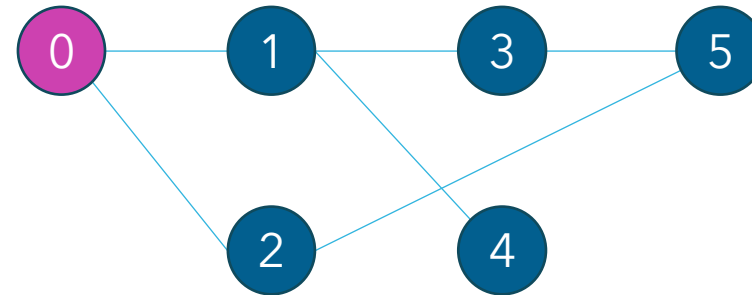
	0	1	2	3	4	5
dist	0	1	1	2	2	∞
visited	1	1	1	1	1	0
queue	2	3	4			

- Επειδή το βάρος είναι μη-αρνητικό και κοινό σε όλες τις ακμές, σημασία έχει ο αριθμός των ακμών που συνιστούν το μονοπάτι, άρα μπορούμε να εκτελέσουμε τον αλγόριθμο BFS, με πολυπλοκότητα $O(|V| + |E|)$.
 - δίνουμε αριθμητική ετικέτα σε κάθε κόμβο ($s \Rightarrow 0, 1, \dots, |V|$)
 - σε έναν πίνακα (dist), μήκους $|V|$, που αρχικοποιείται με τιμές, 0 για τον s και $+\infty$ για τους υπόλοιπους, αποθηκεύουμε το κόστος του συντομότερου μονοπατιού για εκάστοτε κόμβο (από τον s προς κάθε άλλο)
 - αρχικοποιούμε μια ουρά (queue), στην οποία τοποθετούμε τον κόμβο s
 - σε έναν πίνακα (visited), μήκους $|V|$, που αρχικοποιείται με τιμές 0 (εκτός του s , που είναι 1), αποθηκεύουμε την πληροφορία επίσκεψης της κάθε κορυφής (0 ή 1)
 - Ενώσω η ουρά δεν είναι κενή:
 - αφαιρούμε έναν κόμβο x από την ουρά
 - για κάθε γειτονικό του κόμβο, u , αν δεν έχουμε επισκεφθεί τον u
 - $\text{dist}[u] = \text{dist}[x] + 1$ (γιατί $\text{dist}[u] > \text{dist}[x] + 1$)
 - τοποθέτησε τον u στην ουρά
 - $\text{visited}[u] = 1$



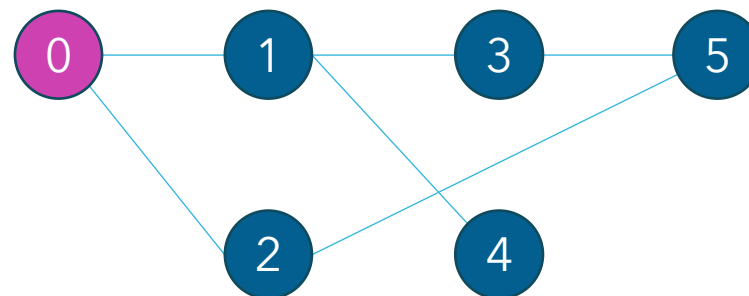
	0	1	2	3	4	5
dist	0	1	1	2	2	2
visited	1	1	1	1	1	1
queue	3	4	5			

- Επειδή το βάρος είναι μη-αρνητικό και κοινό σε όλες τις ακμές, σημασία έχει ο αριθμός των ακμών που συνιστούν το μονοπάτι, άρα μπορούμε να εκτελέσουμε τον αλγόριθμο BFS, με πολυπλοκότητα $O(|V| + |E|)$.
 - δίνουμε αριθμητική ετικέτα σε κάθε κόμβο ($s \Rightarrow 0, 1, \dots, |V|$)
 - σε έναν πίνακα (`dist`), μήκους $|V|$, που αρχικοποιείται με τιμές, 0 για τον s και $+\infty$ για τους υπόλοιπους, αποθηκεύουμε το κόστος του συντομότερου μονοπατιού για εκάστοτε κόμβο (από τον s προς κάθε άλλο)
 - αρχικοποιούμε μια ουρά (`queue`), στην οποία τοποθετούμε τον κόμβο s
 - σε έναν πίνακα (`visited`), μήκους $|V|$, που αρχικοποιείται με τιμές 0 (εκτός του s , που είναι 1), αποθηκεύουμε την πληροφορία επίσκεψης της κάθε κορυφής (0 ή 1)
 - Ενώσω η ουρά δεν είναι κενή:
 - αφαιρούμε έναν κόμβο x από την ουρά
 - για κάθε γειτονικό του κόμβο, u , αν δεν έχουμε επισκεφθεί τον u
 - $\text{dist}[u] = \text{dist}[x] + 1$ (γιατί $\text{dist}[u] > \text{dist}[x] + 1$)
 - τοποθέτησε τον u στην ουρά
 - $\text{visited}[u] = 1$



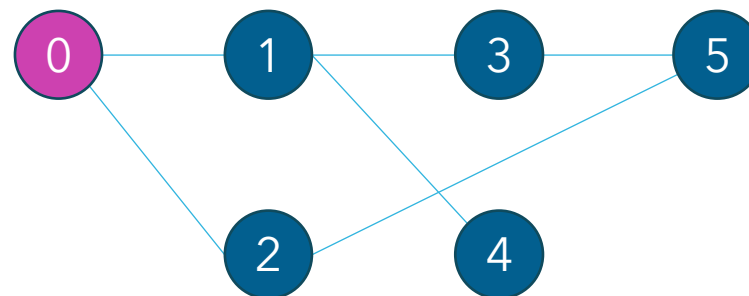
	0	1	2	3	4	5
dist	0	1	1	2	2	2
visited	1	1	1	1	1	1
queue	4	5				

- Επειδή το βάρος είναι μη-αρνητικό και κοινό σε όλες τις ακμές, σημασία έχει ο αριθμός των ακμών που συνιστούν το μονοπάτι, άρα μπορούμε να εκτελέσουμε τον αλγόριθμο BFS, με πολυπλοκότητα $O(|V| + |E|)$.
 - δίνουμε αριθμητική ετικέτα σε κάθε κόμβο ($s \Rightarrow 0, 1, \dots, |V|$)
 - σε έναν πίνακα (dist), μήκους $|V|$, που αρχικοποιείται με τιμές, 0 για τον s και $+\infty$ για τους υπόλοιπους, αποθηκεύουμε το κόστος του συντομότερου μονοπατιού για εκάστοτε κόμβο (από τον s προς κάθε άλλο)
 - αρχικοποιούμε μια ουρά (queue), στην οποία τοποθετούμε τον κόμβο s
 - σε έναν πίνακα (visited), μήκους $|V|$, που αρχικοποιείται με τιμές 0 (εκτός του s , που είναι 1), αποθηκεύουμε την πληροφορία επίσκεψης της κάθε κορυφής (0 ή 1)
 - Ενώσω η ουρά δεν είναι κενή:
 - αφαιρούμε έναν κόμβο x από την ουρά
 - για κάθε γειτονικό του κόμβο, u , αν δεν έχουμε επισκεφθεί τον u
 - $\text{dist}[u] = \text{dist}[x] + 1$ (γιατί $\text{dist}[u] > \text{dist}[x] + 1$)
 - τοποθέτησε τον u στην ουρά
 - $\text{visited}[u] = 1$



	0	1	2	3	4	5
dist	0	1	1	2	2	2
visited	1	1	1	1	1	1
queue	5					

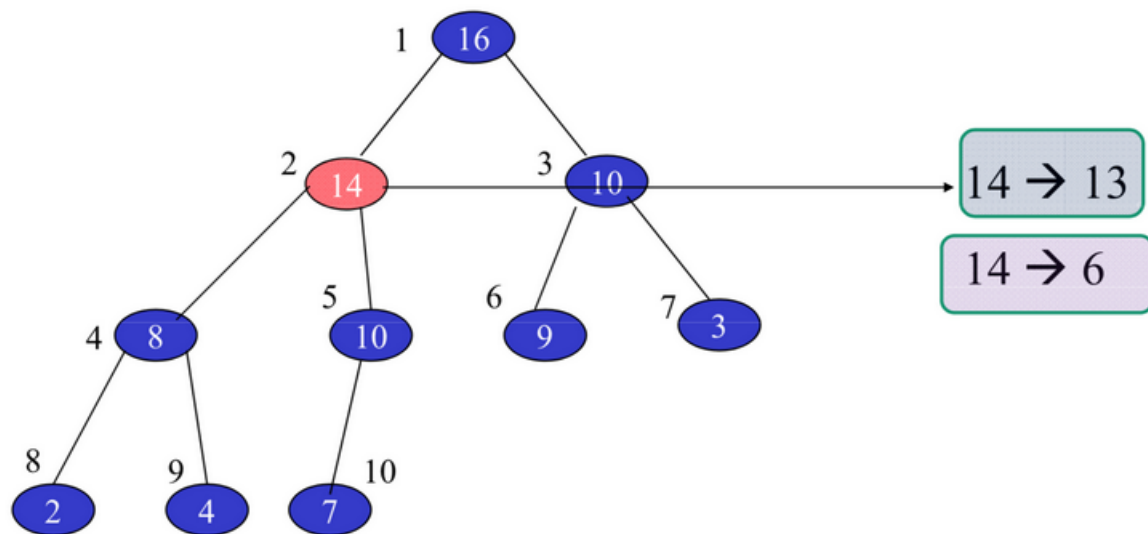
- Επειδή το βάρος είναι μη-αρνητικό και κοινό σε όλες τις ακμές, σημασία έχει ο αριθμός των ακμών που συνιστούν το μονοπάτι, άρα μπορούμε να εκτελέσουμε τον αλγόριθμο BFS, με πολυπλοκότητα $O(|V| + |E|)$.
 - δίνουμε αριθμητική ετικέτα σε κάθε κόμβο ($s \Rightarrow 0, 1, \dots, |V|$)
 - σε έναν πίνακα (dist), μήκους $|V|$, που αρχικοποιείται με τιμές, 0 για τον s και $+\infty$ για τους υπόλοιπους, αποθηκεύουμε το κόστος του συντομότερου μονοπατιού για εκάστοτε κόμβο (από τον s προς κάθε άλλο)
 - αρχικοποιούμε μια ουρά (queue), στην οποία τοποθετούμε τον κόμβο s
 - σε έναν πίνακα (visited), μήκους $|V|$, που αρχικοποιείται με τιμές 0 (εκτός του s , που είναι 1), αποθηκεύουμε την πληροφορία επίσκεψης της κάθε κορυφής (0 ή 1)
 - Ενώσω η ουρά δεν είναι κενή:
 - αφαιρούμε έναν κόμβο x από την ουρά
 - για κάθε γειτονικό του κόμβο, u , αν δεν έχουμε επισκεφθεί τον u
 - $\text{dist}[u] = \text{dist}[x] + 1$ (γιατί $\text{dist}[u] > \text{dist}[x] + 1$)
 - τοποθέτησε τον u στην ουρά
 - $\text{visited}[u] = 1$



	0	1	2	3	4	5
dist	0	1	1	2	2	2
visited	1	1	1	1	1	1
queue						

(B) Η ακολουθία $a_1, a_2, a_3, \dots, a_n$ είναι αποθηκευμένη στο μονοδιάστατο πίνακα A υπό δομή σωρού (*max heap*). Κάποιος όρος a_i αλλάζει και παίρνει μικρότερη τιμή. Ο νέος πίνακας A ενδέχεται να μην είναι πλέον σωρός.

- Na δοθεί σύντομα ένας αναδρομικός αλγόριθμος $RA(A, i)$ που διατηρεί στον A τη δομή σωρού.
- Na δοθεί η αναδρομική σχέση $T(n)$ που περιγράφει τη πολυπλοκότητα του αλγορίθμου στη χειρίστη περίπτωση.
- Na επιλυθεί η αναδρομική εξίσωση $T(n)$, $T(1) = \Theta(1)$ για την εύρεση της πολυπλοκότητας.
- Θεωρήστε το στιγμιότυπο στην παρακάτω εικόνα όπου ο πίνακας A περιγράφεται από ένα δυαδικό δέντρο. Σε κάθε κόμβο i του δέντρου εμφανίζεται εσωτερικά η τιμή a_i . Εφαρμόστε τον αλγόριθμο για τον όρο a_2 όταν αλλάζει τιμή από 14 σε 13 και από 14 σε 6.



```

4 def heapify(heap, index):
5
6     heap_size = len(heap)
7     child_indices = np.array([2*index+1, 2*index+2])
8     child_exist = [c_idx < heap_size for c_idx in child_indices]
9
10    # If index is a leaf, stop
11    if any(child_exist):
12
13        # Get the index of the child with the largest value
14        largest_index = child_indices[np.argmax(heap[child_indices[child_exist]])]
15
16        # If the largest is not the current index, swap and continue heapifying
17        if heap[largest_index] > heap[index]:
18
19            heap[index], heap[largest_index] = heap[largest_index], heap[index]
20            heap = heapify(heap, largest_index)
21
22    return(heap)

```

- Στη χειρότερη περίπτωση, η τιμή της ρίζας θα γίνει τόσο μικρή ώστε να αποτελεί τελικά φύλλο. Άρα, θα κατέβει $\log(n)$ επίπεδα, με κόστος για καθένα $O(1)$. Έχοντας στο αρχικό μας δέντρο n κόμβους, ένα υποδένδρο πλήρους δυαδικού δένδρου θα έχει το πολύ $2n/3$ κόμβους, με το επίπεδο των φύλλων να περιλαμβάνεται εξολοκλήρου σε αυτό το υποδένδρο.

$$T(n) \leq T\left(\frac{n}{3/2}\right) + O(1)$$

$$a = 1 \geq 1, b = 1.5 > 1, f(n) = c > 0$$

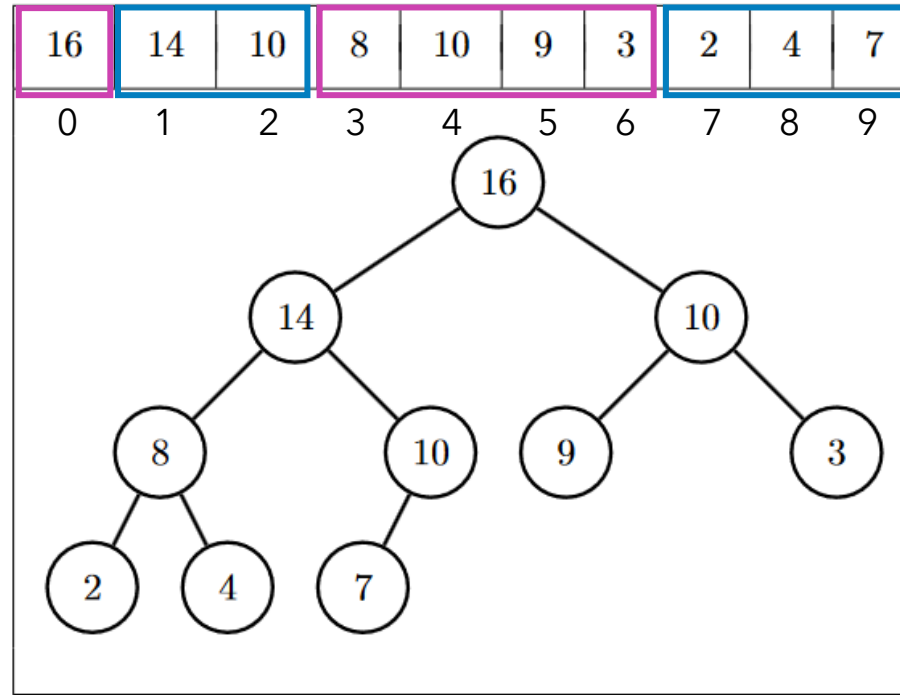
$$n^{\log_b a} = n^{\log_{1.5} 1} = n^0 = 1$$

$$f(n) = c = \Theta(1)$$

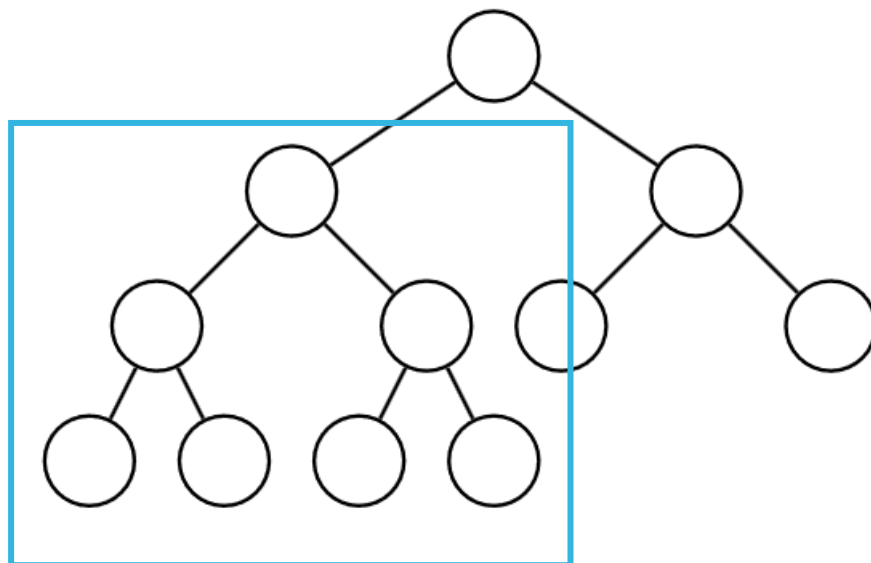
$$\Pi 2: T(n) = O(\log n)$$

στοιχεία κάθε επιπέδου,
από τη ρίζα έως τα φύλλα:

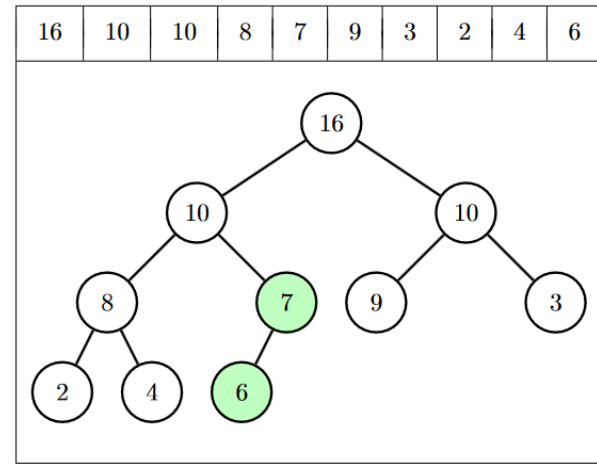
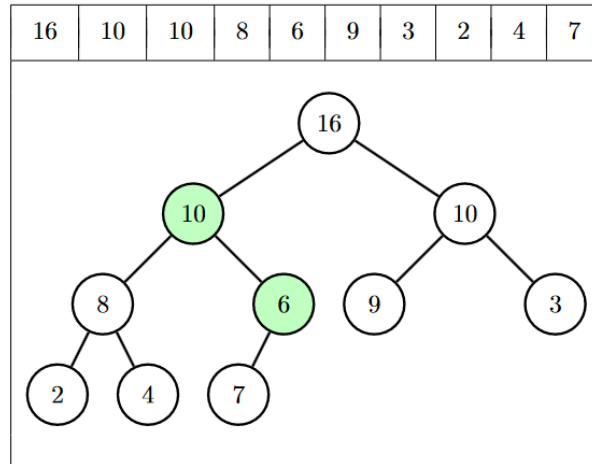
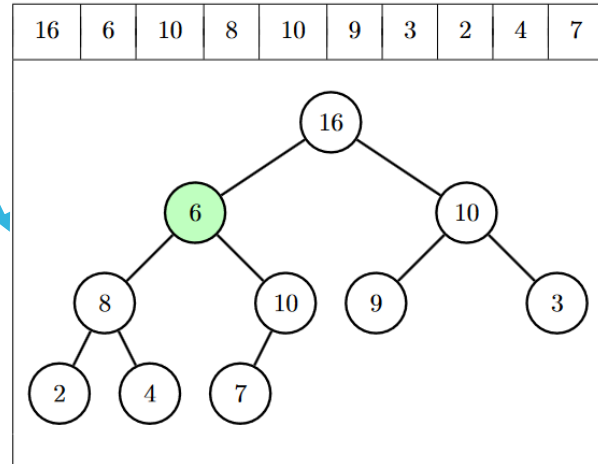
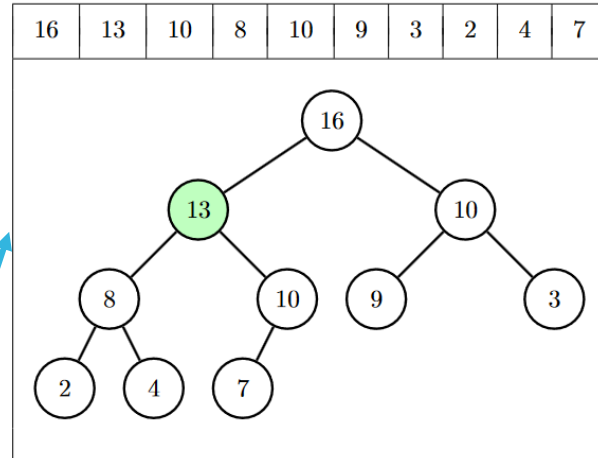
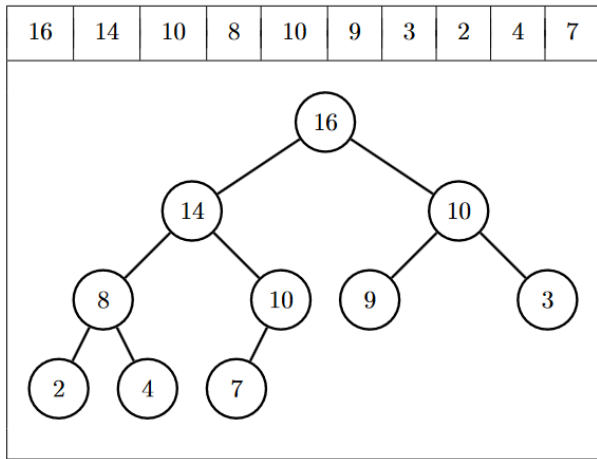
2^0 2^1 2^2 φύλλα: έως και $2^3 = 2^h$



- node = i (0-based)
- left child = $2*i + 1$
- right child = $2*i + 2$

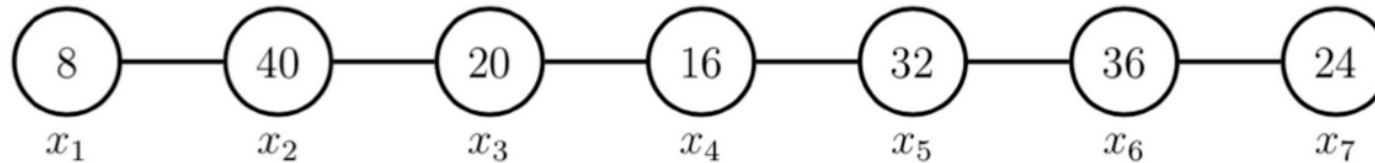


- $n = \sum_{i=0}^{h-1} 2^i + \frac{2^h}{2} = 2^h - 1 + \frac{2^h}{2} = \frac{3}{2} 2^h - 1$
- $|\text{αριστερό υποδένδρο}| = \frac{\sum_{i=1}^{h-1} 2^i}{2} + \frac{2^h}{2} = \frac{2^h - 2}{2} + \frac{2^h}{2} = 2^h - 1$
- $\frac{2^h - 1}{\frac{3}{2} 2^{h-1}} \cong \frac{1}{\frac{3}{2}} = \frac{2}{3}$



Ο Δήμος μιας πόλης θέλει να εγκαταστήσει κολώνες φωτισμού κατά μήκος ενός νέου δρόμου που έχει πρόσφατα κατασκευάσει, σε συγκεκριμένες θέσεις επί των πεζοδρομίων που έχουν υποδειχθεί από τους κατοίκους. Για λόγους εξοικονόμησης κόστους, ο Δήμος έχει αποφασίσει ότι δεν θα τοποθετήσει κολώνες σε δύο διαδοχικές τέτοιες θέσεις, αλλά σε ένα υποσύνολο μη διαδοχικών θέσεων οι οποίες θα εξασφαλίζουν τη συνολικά μέγιστη δυνατή φωτεινότητα.

Μοντελοποιώντας γραφο-θεωρητικά το πρόβλημα, προκύπτει ένα μη κατευθυνόμενο γράφημα $\Gamma = (K, A)$, όπου $K = \{x_1, \dots, x_n\}$ είναι ένα σύνολο n κορυφών (μία για κάθε δυνητική θέση τοποθέτησης κολώνας), $A = \{(x_i, x_{i+1}) : 1 \leq i < n\}$ ένα σύνολο ακμών, και κάθε κορυφή x_i συσχετίζεται με την φωτεινότητα ϕ_i που εξασφαλίζεται από τη συγκεκριμένη θέση. Ένα υποσύνολο κορυφών ονομάζεται ανεξάρτητο αν οποιεσδήποτε δύο κορυφές του Π δεν συνδέονται με ακμή, και ονομάζεται μέγιστο ανεξάρτητο υποσύνολο αν αποφέρει τη μέγιστη συνολικά φωτεινότητα μεταξύ όλων των υποσυνόλων του K . Στόχος λοιπόν είναι να βρεθεί το μέγιστο ανεξάρτητο υποσύνολο, αφού η εξοικονόμηση κόστους δεν επιτρέπει την τοποθέτηση κολώνων φωτισμού σε δύο διαδοχικές θέσεις, δηλαδή σε δύο γειτονικές κορυφές του Γ . Θεωρήστε, για παράδειγμα, το παρακάτω στιγμιότυπο 7 κορυφών στο οποίο οι αριθμοί εντός των κορυφών συμβολίζουν τη φωτεινότητα.

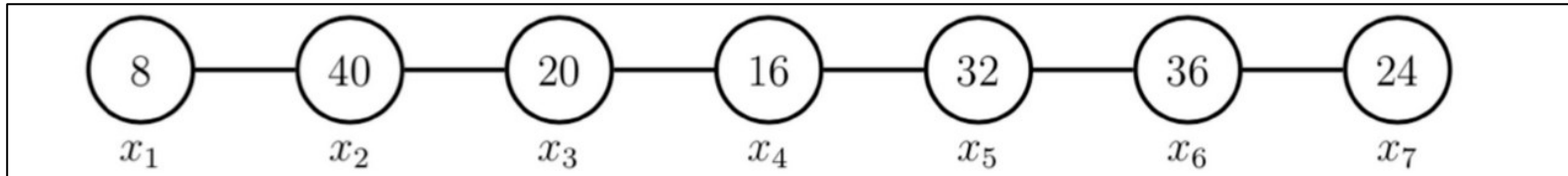


Τα υποσύνολα $\Pi_1 = \{x_1, x_3, x_5, x_7\}$, $\Pi_2 = \{x_2, x_4, x_6\}$, $\Pi_3 = \{x_2, x_5, x_7\}$ και $\Pi_4 = \{x_1, x_4, x_7\}$ είναι ανεξάρτητα, με συνολική φωτεινότητα 84, 92, 96 και 48 αντίστοιχα. Το Π_3 είναι το μέγιστο ανεξάρτητο υποσύνολο για το παραπάνω στιγμιότυπο 7 κορυφών και αποφέρει τη συνολικά μέγιστη δυνατή φωτεινότητα.

1. Θεωρήστε τον εξής άπληστο αλγόριθμο επίλυσης του παραπάνω προβλήματος: επιλέγει το υποσύνολο κορυφών με περιττούς δείκτες και το υποσύνολο κορυφών με άρτιους δείκτες, και επιστρέφει ως λύση

εκείνο το υποσύνολο με τη συνολικά μέγιστη φωτεινότητα. Είναι ο άπληστος αυτός αλγόριθμος βέλτιστος; Αν ναι, αποδείξτε ότι βρίσκει πάντοτε την βέλτιστη λύση. Αν όχι, δώστε ένα αντιπαράδειγμα για το οποίο ο αλγόριθμος βρίσκει μια μη βέλτιστη λύση.

2. Σχεδιάστε έναν αλγόριθμο δυναμικού προγραμματισμού ο οποίος, δεδομένου ενός στιγμιότυπου του παραπάνω προβλήματος, βρίσκει τη συνολική φωτεινότητα του μέγιστου ανεξάρτητου υποσυνόλου. Η περιγραφή του αλγορίθμου μπορεί να είναι σε άτυπη μορφή (φυσική γλώσσα), αλλά πρέπει να περιλαμβάνει οπωσδήποτε την/τις αναδρομική/-κες σχέση/-εις που διέπουν τον αλγόριθμο και συμπληρώνουν τον πίνακα δυναμικού προγραμματισμού.
3. Δώστε τον χρόνο εκτέλεσης του αλγορίθμου σας, ο οποίος πρέπει να είναι πολυωνυμικός ως προς το n και ανεξάρτητος της τιμής φωτεινότητας κάθε κολώνας.
4. Εκτελέστε τον αλγόριθμό σας στο παραπάνω παράδειγμα δίνοντας τις τιμές του πίνακα δυναμικού προγραμματισμού σε κάθε βήμα.



Ο άπληστος αλγόριθμος, δεν είναι βέλτιστος, καθώς υπάρχει το αντιπαράδειγμα που προκύπτει από το δοθέν στιγμιότυπο:

- Για τις θέσεις με περιττό δείκτη:
 - άθροισμα φωτεινότητας = $x_1 + x_3 + x_5 + x_7 = 8 + 20 + 32 + 24 = 84$
- Για τις θέσεις με άρτιο δείκτη:
 - άθροισμα φωτεινότητας = $x_2 + x_4 + x_6 = 40 + 16 + 36 = 92$
- Ο αλγόριθμος θα επέστρεφε ως μέγιστη αθροιστική φωτεινότητα την τιμή 92, όμως το υποσύνολο $\{x_2, x_5, x_7\}$, έχει μεγαλύτερη αθροιστική φωτεινότητα = 96.

(Α') Έστω Π_i το μέγιστο ανεξάρτητο σύνολο στη διαδρομή $x_1 - x_i$ και έστω ότι $\Phi[i]$ συμβολίζει τη συνολική φωτεινότητα του Π_i . Ορίζουμε $\Phi[0] = 0$. Προφανώς, $\Phi[1] = \phi_1$.

Για $i > 1$, ισχύει ότι είτε η x_i ανήκει στο Π_i , είτε δεν ανήκει. Στην πρώτη περίπτωση ($x_i \in \Pi_i$), γνωρίζουμε ότι η x_{i-1} δεν μπορεί να ανήκει στο Π_i (αφού είναι ανεξάρτητο υποσύνολο) και άρα $\Phi[i] = \phi_i + \Phi[i-2]$. Στη δεύτερη περίπτωση ($x_i \notin \Pi_i$), έχουμε ότι $\Phi[i] = \Phi[i-1]$.

Επομένως, προκύπτει η αναδρομική σχέση $\Phi[i] = \max\{\Phi[i-1], \phi_i + \Phi[i-2]\}$.

Μπορούμε να υπολογίσουμε τις τιμές του πίνακα δυναμικού προγραμματισμού Φ σε αυξανόμενη σειρά ως προς το $i = 1, \dots, n$. Η συνολική φωτεινότητα του μέγιστου ανεξάρτητου υποσυνόλου είναι η τιμή του $\Phi[n]$.

(Β') (i) Ο χρόνος εκτέλεσης του αλγορίθμου είναι $O(n)$, αφού αποτελείται από n επαναλήψεις και σε κάθε επανάληψη εκτελούμε ένα σταθερό αριθμό πράξεων.

(ii) Η εκτέλεση του αλγορίθμου στο παράδειγμα έχει ως εξής:

- $\Phi[0] = 0$
- $\Phi[1] = 8$
- $\Phi[2] = \max\{\Phi[1], \phi_2 + \Phi[0]\} = \max\{8, 40 + 0\} = 40$.
- $\Phi[3] = \max\{\Phi[2], \phi_3 + \Phi[1]\} = \max\{40, 20 + 8\} = 40$.
- $\Phi[4] = \max\{\Phi[3], \phi_4 + \Phi[2]\} = \max\{40, 16 + 40\} = 56$.
- $\Phi[5] = \max\{\Phi[4], \phi_5 + \Phi[3]\} = \max\{56, 32 + 40\} = 72$.
- $\Phi[6] = \max\{\Phi[5], \phi_6 + \Phi[4]\} = \max\{72, 36 + 56\} = 92$.
- $\Phi[7] = \max\{\Phi[6], \phi_7 + \Phi[5]\} = \max\{92, 24 + 72\} = 96$.

Άρα, το μέγιστο ανεξάρτητο υποσύνολο έχει συνολική φωτεινότητα 96.

CLIQUE: Δοθέντος ενός μη-κατευθυνόμενου γράφου G με n κόμβους και ενός μη-αρνητικού ακεραίου $k \leq n$, περιέχει ο G **k -κλίκα**;

Μια **k -κλίκα** είναι ένας πλήρως συνδεδεμένος γράφος με k κόμβους και $k(k-1)/2$ ακμές.

- i. Αποδείξτε ότι μια λύση για το πρόβλημα **CLIQUE** μπορεί να επαληθευθεί σε πολυωνυμικό χρόνο (= ανήκει στην κλάση NP).
- ii. Αποδείξτε ότι όταν το k έχει σταθερή τιμή, π.χ. $k = 100$, τότε το πρόβλημα **CLIQUE** λύνεται σε πολυωνυμικό χρόνο (= ανήκει στην κλάση P). Να περιγραφεί σύντομα, σε φυσική γλώσσα, ο αλγόριθμος και να δοθεί η πολυπλοκότητα.

i. Μια δοθείσα λύση (σύνολο κορυφών), μπορεί να επαληθευθεί σε πολυωνυμικό χρόνο ως εξής:

- ελέγχεται αν η πληθικότητα του συνόλου κορυφών που δίνεται είναι k , σε $O(k)$. Αν δεν είναι, τότε δεν πρόκειται για σωστή λύση.
- ελέγχουμε αν υπάρχουν όλες οι απαραίτητες ακμές, $\frac{k(k-1)}{2} = O(k^2)$
- αν υπάρχουν όλες οι ακμές, τότε η δοθείσα λύση είναι σωστή.
- συνολική πολυπλοκότητα: $\max\{O(k), O(k^2)\} = O(k^2) = O(n^2)$

Άρα, ανήκει στην κλάση NP.

ii. Αυτό, μπορεί να λυθεί με πολυωνυμικό αλγόριθμο ως εξής:

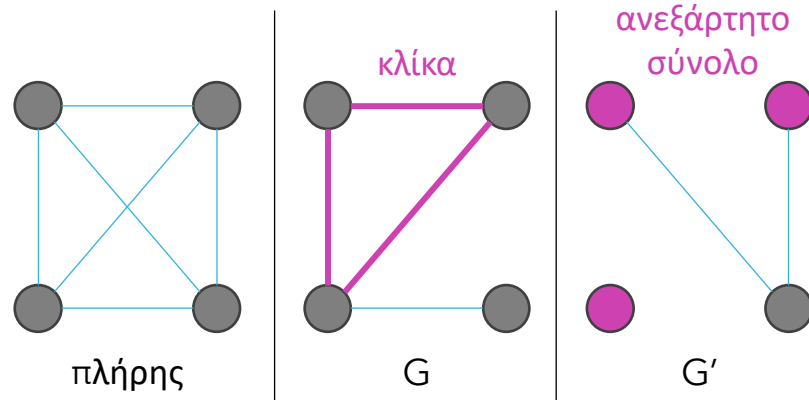
- βρίσκουμε όλα τα δυνατά υποσύνολα k κόμβων $= \binom{n}{k} = \frac{n!}{k!(n-k)!} = \frac{n(n-1)\dots(n-k+1)}{k!} = \Theta(n^k)$
- για κάθε υποσύνολο, ελέγχουμε αν υπάρχουν όλες οι απαραίτητες ακμές, $\frac{k(k-1)}{2} = O(k^2)$
- αν υπάρχουν όλες οι ακμές για κάποιο υποσύνολο κόμβων, τότε υπάρχει k -κλίκα, διαφορετικά δεν υπάρχει.
- συνολική πολυπλοκότητα: $O(n^k k^2) \xrightarrow{k=100} O(n^{100} 100^2) = O(n^{100})$

INDEP: Δοθέντος ενός μη-κατευθυνόμενου γράφου G με n κόμβους και ενός μη-αρνητικού ακεραίου $k \leq n$, περιέχει ο G **k -ανεξάρτητο σύνολο**;

Ένα **k -ανεξάρτητο σύνολο** είναι k κόμβοι που ανά 2 δε συνδέονται με ακμή.

- i. Αποδείξτε ότι μια λύση για το πρόβλημα **INDEP** μπορεί να επαληθευθεί σε πολυωνυμικό χρόνο (= ανήκει στην κλάση NP).
- ii. Αποδείξτε ότι όταν το k έχει σταθερή τιμή, π.χ. $k = 1000$, τότε το πρόβλημα **INDEP** λύνεται σε πολυωνυμικό χρόνο (= ανήκει στην κλάση P). Να περιγραφεί σύντομα, σε φυσική γλώσσα, ο αλγόριθμος και να δοθεί η πολυπλοκότητα.

συμπληρωματικός / αντίστροφος γράφος:
ακμές πλήρους γράφου με τις ίδιες κορυφές - ακμές G



i. Μια δοθείσα λύση (σύνολο κορυφών), μπορεί να επαληθευθεί σε πολυωνυμικό χρόνο ως εξής:

- ελέγχεται αν η πληθικότητα του συνόλου κορυφών που δίνεται είναι k , σε $O(k)$. Αν δεν είναι, τότε δεν πρόκειται για σωστή λύση.
- ελέγχουμε αν υπάρχει κάποια ακμή από τις, $\frac{k(k-1)}{2} = O(k^2)$, που θα καθιστούσαν το σύνολο μη-ανεξάρτητο και θα ανήκαν στην αντίστοιχη k -κλίκα.
- αν υπάρχει κάποια τέτοια ακμή, τότε η δοθείσα λύση είναι λαθεμένη, διαφορετικά είναι σωστή.
- συνολική πολυπλοκότητα: $\max\{O(k), O(k^2)\} = O(k^2) = O(n^2)$

Άρα, ανήκει στην κλάση NP.

ii. Αυτό μπορεί να λυθεί με πολυωνυμικό αλγόριθμο ως εξής:

- βρίσκουμε όλα τα δυνατά υποσύνολα k κόμβων $= \binom{n}{k} = \frac{n!}{k!(n-k)!} = \frac{n(n-1)\dots(n-k+1)}{k!} = \Theta(n^k)$
- για κάθε υποσύνολο, ελέγχουμε αν υπάρχει κάποια ακμή από τις, $\frac{k(k-1)}{2} = O(k^2)$, που θα καθιστούσαν το σύνολο μη-ανεξάρτητο και θα ανήκαν στην αντίστοιχη k -κλίκα.
- αν υπάρχει κάποια τέτοια ακμή για όλα τα υποσύνολα κόμβων, δεν υπάρχει k -ανεξάρτητο σύνολο, διαφορετικά υπάρχει.
- συνολική πολυπλοκότητα: $O(n^k k^2) \xrightarrow{k=1000} O(n^{1000} 1000^2) = O(n^{1000})$

Θεωρήστε τα προβλήματα:

- ελαχιστοποίηση μονοπατιού ανάμεσα σε 2 κόμβους s, t ενός γράφου, και
 - μεγιστοποίηση οφέλους ενός 0-1 σακιδίου (διακριτού σακιδίου).
- i. Να δοθούν τα αντίστοιχα προβλήματα απόφασης **D(Path)** και **D(K)**.
 - ii. Αποδείξτε ότι μια λύση για τα προβλήματα αυτά μπορεί να επαληθευθεί σε πολυωνυμικό χρόνο (= ανήκουν στην κλάση NP).
 - iii. Αποδείξτε ότι το $D(K)$ λύνεται σε πολυωνυμικό χρόνο (= ανήκει στην κλάση P).

i.

D(Path): Υπάρχει στον γράφο μονοπάτι μεταξύ των s, t με συνολικό κόστος $\leq k$;

D(K): Υπάρχει επιλογή αντικειμένων με συνολικό κόστος $\geq k$;

ii,iii.

D(Path): P, NP

Υπάρχει πολυωνυμικός αλγόριθμος για την επαλήθευση δοθείσας λύσης, $O(n)$:

Για κάθε ακμή της δοθείσας λύσης, αθροίζονται τα κόστη και ελέγχεται αν το άθροισμα είναι $\leq k$.

Υπάρχει πολυωνυμικός αλγόριθμος για την επίλυση π.χ. με τον αλγόριθμο Bellman-Ford (BF) σε χρόνο $O(n^3)$, όταν δεν έχουμε αρνητικού κόστους κύκλο αλλά με τη δυνατότητα ύπαρξης αρνητικών βαρών.

D(K): NP

Υπάρχει πολυωνυμικός αλγόριθμος για την επαλήθευση δοθείσας λύσης, $O(n)$:

Για τα αντικείμενα της λύσης, αθροίζονται αφενός τα βάρη τους και ελέγχεται ότι το άθροισμα δεν υπερβαίνει τη χωρητικότητα του σακιδίου και αφετέρου τα κόστη και ελέγχεται ότι το άθροισμά τους είναι $\geq k$.

Θεωρήστε τα προβλήματα:

- ελαχιστοποίηση κόστους ενός δέντρου επικάλυψης (MST) σε έναν γράφο, και
- ελαχιστοποίησης του κόστους ενός χαμιλτονιανού κύκλου (TSP) σε έναν πλήρη γράφο.

- i. Να δοθούν τα αντίστοιχα προβλήματα απόφασης **D(MST)** και **D(TSP)**.
- ii. Αποδείξτε ότι μια λύση για τα προβλήματα αυτά μπορεί να επαληθευθεί σε πολυωνυμικό χρόνο (= ανήκουν στην κλάση NP).
- iii. Αποδείξτε ότι το D(MST) λύνεται σε πολυωνυμικό χρόνο (= ανήκει στην κλάση P).

i.

D(ST): Υπάρχει δέντρο επικάλυψης με συνολικό κόστος $\leq k$;

D(TSP): Υπάρχει χαμιλτονιανός κύκλος σε πλήρη γράφο με συνολικό κόστος $\leq k$;

ii,iii.

D(ST): P, NP

Υπάρχει πολυωνυμικός αλγόριθμος για την επαλήθευση δοθείσας λύσης:

Για κάθε ακμή της δοθείσας λύσης, αθροίζονται τα κόστη και ελέγχεται αν το άθροισμα είναι $\leq k$. Επίσης, ελέγχεται ότι το σύνολο των κορυφών που αντιστοιχούν στις ακμές περιλαμβάνει όλες του αρχικού γράφου και επίσης ότι δεν υπάρχει κύκλος (π.χ. DFS, με $O(k^2) = O(n^2)$).

Υπάρχει πολυωνυμικός αλγόριθμος για την εύρεση του MST, π.χ. με χρήση του Kruskal σε χρόνο $O(n^3)$ και σύγκριση της τιμής k με το άθροισμα βαρών των ακμών που επιστρέφει.

D(TSP): NP

Υπάρχει πολυωνυμικός αλγόριθμος για την επαλήθευση δοθείσας λύσης, $O(n)$:

Ελέγχεται ότι αποτελεί μονοπάτι και με πληθικότητα των ακμών, n , ότι περιλαμβάνονται όλες οι κορυφές 1 μόνο φορά σε αυτές τις ακμές (εκτός της αρχής/τέλους), ότι η πρώτη κορυφή είναι και η τελευταία και ότι το άθροισμα των βαρών των ακμών είναι $\leq k$.

Θέλουμε να υπολογιστεί με αναδρομικό αλγόριθμο, η ακολουθία $a_1, a_2, \dots, a_n$, που προκύπτει από τον αναδρομικό τύπο:

$$a_n = 2 \max\{a_{n/2}, a_{n/2+1}\} + a_{n/2-1}$$

με τους 3 αρχικούς όρους να παίρνουν τιμή 1. Θεωρήστε ότι $i/2$ είναι το κάτω μέρος του ρητού $i/2$. $\log_2(3) = 1.585$. Έστω **RA(n)** ο αλγόριθμος που στηρίζεται στην αναδρομική σχέση και στις αρχικές τιμές.

- i. Γράψτε σε φυσική γλώσσα και σύντομα τον αλγόριθμο **RA(n)**.
- ii. Δείξτε ότι ο **RA(n)** είναι πολυωνυμικός αλγόριθμος με πολυπλοκότητα $O(n^{1.585})$.
- iii. Αν χρησιμοποιήσουμε για τον υπολογισμό δυναμικό προγραμματισμό (αλγόριθμος **DA(n)**), πόσα υποπροβλήματα θα οριστούν;
- iv. Δικαιολογήστε ότι η πολυπλοκότητα του **DA(n)** είναι γραμμική.
- v. Ποιος αλγόριθμος είναι ταχύτερος;

- i.
- $RA(n)$, $n > 0$
 - αν $n \leq 3$:
 - επίστρεψε (1)
 - αλλιώς:
 - $num1 = RA(\text{floor}(n/2))$
 - $num2 = RA(\text{floor}(n/2)+1)$
 - $num3 = RA(\text{floor}(n/2)-1)$
 - επίστρεψε ($2 * \max(num1, num2) + num3$)
- iii.
- $DA(n)$, $n > 0$
 - $a = [1, \dots, 1]$ // πίνακας n θέσεων με τιμή 1 σε καθεμία
 - για i από 4 έως n :
 - $a[i] = 2 * \max(a[\text{floor}(i/2)], a[\text{floor}(i/2)+1]) + a[\text{floor}(i/2)-1]$
 - επίστρεψε($a[n]$)
- Ορίζονται n υποπροβλήματα, και για τα 3 πρώτα, γνωρίζουμε εξ αρχής τις τιμές.
- iv. Με τον $DA(n)$, υπολογίζουμε μία φορά την τιμή κάθε θέσης του πίνακα a , μεγέθους n , και χρησιμοποιούμε τιμές που έχουν ήδη υπολογιστεί / είναι γνωστές, με πράξεις (πρόσθεση, πολλαπλασιασμός, εύρεση μέγιστης τιμής ανάμεσα σε δύο αριθμούς) που θεωρούμε πως έχουν πολυπλοκότητα $\Theta(1)$.
- v. Εφόσον ο $DA(n)$ είναι τάξης $\Theta(n)$ και ο $RA(n)$ είναι τάξης $\Theta(n^{1.585})$, προκύπτει άμεσα ότι ο $DA(n)$ είναι ταχύτερος.

ii. Ο χρόνος εύρεσης του a_n , ισούται με τον χρόνο εύρεσης 3 αριθμών μεγέθους περίπου $n/2$ συν τον χρόνο διάσπασης και συνδυασμού των λύσεων.

Αυτό, οδηγεί στην αναδρομική εξίσωση:

$$T(n) = 3T(n/2) + O(n)$$

$$a = 3 \geq 1, b = 2 \geq 2$$

$$f(n) = O(n) = O(n^{\log_2(3) - 0.5}) = O(n^{1.585 - 0.5}) = O(n^{1.085})$$

$$\text{Από την Π1 του θεωρήματος κυριαρχίας: } T(n) = \Theta(n^{1.585}) = O(n^{1.585})$$

Με επαγωγή, θδο η ακολουθία είναι αύξουσα:

$$\alpha_1 = \alpha_2 = \alpha_3 = 1$$

$$\alpha_4 = 2 \max\{\alpha_2, \alpha_3\} + \alpha_1 = 2\alpha_3 + \alpha_1 = 2\alpha_{4/2+1} + \alpha_{4/2-1} = 2 + 1 = 3$$

- υπόθεση: $\alpha_n \geq \alpha_{n-1}$
- βάση: για κάθε $k \leq 4$: $\alpha_4 = 4 \geq \alpha_3 = 1 \geq \alpha_2 = 1 \geq \alpha_1 = 1$
- βήμα: $\alpha_{k+1} \geq \alpha_k$
 - $2 \max\{\alpha_{(k+1)/2} + \alpha_{(k+1)/2+1}\} + \alpha_{(k+1)/2-1} \geq 2 \max\{\alpha_{k/2} + \alpha_{k/2+1}\} + \alpha_{k/2-1}$
 - επειδή $k/2, k/2+1, (k+1)/2, (k+1)/2+1, (k+1)/2-1 \leq k$ για τιμές ≥ 3 :
 - $\alpha_{(k+1)/2+1} \geq \alpha_{(k+1)/2}, \alpha_{k/2+1} \geq \alpha_{k/2}, \alpha_{(k+1)/2+1} \geq \alpha_{k/2+1}, \alpha_{(k+1)/2-1} \geq \alpha_{k/2-1}$
 - άρα, $2 \alpha_{(k+1)/2+1} + \alpha_{(k+1)/2-1} \geq 2 \alpha_{k/2+1} + \alpha_{k/2-1}$
 - έτσι η υπόθεση ισχύει
- Συνεπώς, θα αρκούσε ο υπολογισμός 2 αριθμών, μεγέθους $\sim n/2$

Θέλουμε να υπολογιστεί με αναδρομικό αλγόριθμο, η ακολουθία $b_1, b_2, \dots, b_n$, που προκύπτει από τον αναδρομικό τύπο:

$$b_n = 2 \max\{b_{n-1}, b_{n-2}\} + b_{n-3}$$

με τους 3 αρχικούς όρους να παίρνουν τιμή 1. Έστω **RB(n)** ο αλγόριθμος που στηρίζεται στην αναδρομική σχέση και στις αρχικές τιμές. Δίνεται ότι $3^{1/3} = 1.44$.

- i. Γράψτε σε φυσική γλώσσα και σύντομα τον αλγόριθμο **RB(n)**.
- ii. Δείξτε ότι ο **RB(n)** είναι εκθετικός αλγόριθμος με πολυπλοκότητα $\Omega(1.44^n)$.
- iii. Αν χρησιμοποιήσουμε για τον υπολογισμό δυναμικό προγραμματισμό (αλγόριθμος **DB(n)**), πόσα υποπροβλήματα θα οριστούν;
- iv. Δικαιολογήστε ότι η πολυπλοκότητα του **DB(n)** είναι γραμμική.
- v. Ποιος αλγόριθμος είναι ταχύτερος;

- i.
- $RB(n)$, $n > 0$
 - αν $n \leq 3$:
 - επίστρεψε (1)
 - αλλιώς:
 - $num1 = RA(n-1)$
 - $num2 = RA(n-2)$
 - $num3 = RA(n-3)$
 - επίστρεψε ($2 * \max(num1, num2) + num3$)
- iii.
- $DB(n)$, $n > 0$
 - $b = [1, \dots, 1]$ // πίνακας n θέσεων με τιμή 1 σε καθεμία
 - για i από 4 έως n :
 - $b[i] = 2 * \max(b[n-1], b[n-2]) + b[n-3]$
 - επίστρεψε($b[n]$)
- Ορίζονται n υποπροβλήματα, και για τα 3 πρώτα, γνωρίζουμε εξαρχής τις τιμές.
- iv. Με τον $DB(n)$, υπολογίζουμε μία φορά την τιμή κάθε θέσης του πίνακα b , μεγέθους n και χρησιμοποιούμε τιμές που έχουν ήδη υπολογιστεί / είναι γνωστές, με πράξεις (πρόσθεση, πολλαπλασιασμός, εύρεση μέγιστης τιμής ανάμεσα σε δύο αριθμούς) που θεωρούμε πως έχουν πολυπλοκότητα $\Theta(1)$.
- iv. Εφόσον ο $DB(n)$ είναι τάξης $\Theta(n)$ και ο $RB(n)$ είναι $\Omega(1.44^n)$, προκύπτει άμεσα ότι ο $DB(n)$ είναι ταχύτερος.

ii.

$$\alpha_1 = \alpha_2 = \alpha_3 = 1$$

$$\alpha_4 = 2 \max\{\alpha_2, \alpha_3\} + \alpha_1 = 2\alpha_3 + \alpha_1 = 2\alpha_{4/2+1} + \alpha_{4/2-1} = 2 + 1 = 3$$

Με επαγωγή, θδο η ακολουθία είναι αύξουσα:

- υπόθεση: $b_n \geq b_{n-1}$
- βάση: για κάθε $k \leq 4$: $\alpha_4 = 3 \geq \alpha_3 = 1 \geq \alpha_2 = 1 \geq \alpha_1 = 1$
- βήμα: $b_{k+1} \geq b_k$
 - $2 \max\{b_k + b_{k-1}\} + b_{k-2} \geq 2 \max\{b_{k-1} + b_{k-2}\} + b_{k-3}$
 - επειδή $k-1, k-2, k-3 \leq k$:
 - $2 b_k + b_{k-2} \geq 2 b_{k-1} + b_{k-3}$
 - άρα η υπόθεση ισχύει
- Συνεπώς, αρκεί ο υπολογισμός 2 αριθμών, των b_{n-1} και b_{n-3} .

Έστω $T(n)$ ο χρόνος υπολογισμού του b_n .

$$3T(n-3) \leq T(n) = 2T(n-1) + T(n-3)$$

$$\begin{aligned} W(n) &= 3W(n-3) \\ &= 3^2W(n-6) \\ &\dots \\ &= 3^kW(n-3k) \\ &\dots \\ &= 3^{n/3}W(0) \\ &= (3^{1/3})^n \Theta(1) = 1.44^n \Rightarrow T(n) = \Omega(1.44^n) \end{aligned}$$

Θεωρήστε το **συνεχές** πρόβλημα του **σακιδίου**:

$$\max \sum_{i=1}^n c_i x_i$$
$$\sum_{i=1}^n a_i x_i \leq b, 0 \leq x_i \leq 1$$

με τις τιμές των $c_i, a_i, 1 \leq i \leq n$, και b να είναι ακέραιοι.

- i. Περιγράψτε σε φυσική γλώσσα έναν άπληστο αλγόριθμο **CONKNAPS(n, c_i, a_i, b)** που επιστρέφει μια **βέλτιστη** λύση.
- ii. Υπολογίστε την πολυπλοκότητα του αλγορίθμου **CONKNAPS(n, c_i, a_i, b)**.
- iii. Θεωρήστε το στιγμιότυπο: οι τιμές των c_i , είναι εν σειρά: 16, 9, 7, 15, 10, 1. Οι τιμές των a_i είναι, αντίστοιχα: 8,5,4,9,6,1 και $b = 12$. Εφαρμόστε τον αλγόριθμο.
- iv. Το **0-1** πρόβλημα του σακιδίου με $x_i \in \{0,1\}$ έχει **πολυωνυμικό αλγόριθμο** που επιστρέφει πάντα τη **βέλτιστη λύση**;
- v. Ποια είναι η πολυπλοκότητα για την εύρεση της βέλτιστης τιμής και ποια η πολυπλοκότητα για την εύρεση της δομής της βέλτιστης λύσης με **δυναμικό** προγραμματισμό στο **0-1** πρόβλημα σακιδίου;

i,ii.

- για όλα τα αντικείμενα, υπολογισμός του λόγου κόστος/βάρος, σε χρόνο $O(n)$
- ταξινόμησή τους με φθίνοντα τρόπο, π.χ. με συγχώνευση, σε χρόνο $O(n \log n)$
- αρχικοποιούμε το συνολικό κέρδος σε 0 και το υπολειπόμενο βάρος του σακιδίου σε b
- διατρέχουμε την ταξινομημένη λίστα των αντικειμένων και για κάθε ένα παίρνουμε τη μεγαλύτερη ποσότητα που είναι δυνατό ώστε η υπολειπόμενη χωρητικότητα του σακιδίου (u_{ch}) να γίνει το πολύ 0, αυξάνοντας κατάλληλα το κόστος και μειώνοντας την u_{ch} , σε χρόνο $O(n)$
- η διαδικασία σταματά όταν η u_{ch} γίνει 0 ή δεν υπάρχουν άλλα αντικείμενα
- επιστρέφεται το κόστος του σακιδίου

Επιστρέφει τη βέλτιστη λύση, αφού με οποιονδήποτε άλλον τρόπο, θα λαμβανόταν ποσότητα μικρότερη από τη μέγιστη δυνατή από αντικείμενο που αυξάνει περισσότερο το κόστος σε αναλογία με το βάρος. Άρα, δεν μπορεί να γίνει καλύτερη επιλογή. Η πολυπλοκότητα της διαδικασίας είναι $\max\{O(n), O(n \log n)\} = O(n \log n)$.

iii.

Διάταξη [(κόστος, βάρος) $\rightarrow$ λόγος]:

(16,8) $\rightarrow$ 2, (9,5) $\rightarrow$ 1.8, (7,4) $\rightarrow$ 1.75, (15,9) $\rightarrow$ 1.67, (10,6) $\rightarrow$ 1.67, (1,1) $\rightarrow$ 1

υπολειπόμενη χωρητικότητα σακιδίου: $res_cap = b = 12$

συνολικό κόστος: $total_cost = 0$

αντικείμενο 1: $res_cap = 12 - 8 = 4$, $total_cost = 0 + 16 = 16$

αντικείμενο 2: $res_cap = 4 - (4/5)5 = 0$, $total_cost = 16 + (4/5)9 = 23.2$

τερματισμός αφού $res_cap = 0$.

επιστροφή $total_cost$.

iv. Το πρόβλημα απόφασης 0-1 του σακιδίου είναι NP-πλήρες, ανάγεται σε αυτό το NP-πλήρες πρόβλημα του αθροίσματος υποσυνόλου και μία λύση που δίνεται (επιλεγμένα αντικείμενα) μπορεί να ελεγχθεί σε γραμμικό χρόνο αν πληροί τον περιορισμό βάρους του σακιδίου και έχει συνολικό κόστος σύμφωνο με τον στόχο, άρα δεν έχει γνωστό πολυωνυμικό αλγόριθμο επίλυσης.

v. Η πολυπλοκότητα για τον υπολογισμό της βέλτιστης τιμής είναι ψευδοπολυωνυμική, $O(n*b)$, αφού ο διδιάστατος πίνακας που δημιουργείται έχει $(n+1)$ γραμμές ($\#αντικειμένων+1$) και $(b+1)$ στήλες (μονάδες βάρους χωρητικότητας σακιδίου+1), και η οπισθοδρόμηση για την εύρεση της δομής έχει γραμμική πολυπλοκότητα ως προς τον αριθμό των αντικειμένων, $O(n)$.

Θεωρήστε το **0-1** πρόβλημα του **σακιδίου**:

$$\max \sum_{i=1}^n c_i x_i$$
$$\sum_{i=1}^n a_i x_i \leq b, x_i \in \{0,1\}$$

με τις τιμές των $c_i, a_i, 1 \leq i \leq n$, και b να είναι ακέραιοι.

- i. Περιγράψτε σε φυσική γλώσσα έναν **άπληστο** αλγόριθμο **KNAPSACK(n, c_i, a_i, b)** που επιστρέφει μια **εφικτή** λύση.
- ii. Υπολογίστε την πολυπλοκότητα του αλγορίθμου **KNAPSACK(n, c_i, a_i, b)**.
- iii. Θεωρήστε το στιγμιότυπο: οι τιμές των c_i , είναι εν σειρά: 16, 9, 7, 15, 10, 1. Οι τιμές των a_i είναι, αντίστοιχα: 8,5,4,9,6,1 και $b = 12$. Εφαρμόστε τον αλγόριθμο.
- iv. Ο αλγόριθμος μπορεί να επιστρέφει πάντα τη βέλτιστη λύση και να είναι πολυωνυμικής πολυπλοκότητας;
- v. Η επίλυση του προβλήματος με δυναμικό προγραμματισμό, πόσα το πλήθος υποπροβλήματα χρειάζεται να ορίσουμε; Να δοθεί η αναδρομική σχέση που συνδέει τη βέλτιστη λύση με τις βέλτιστες λύσεις των υποπροβλημάτων;

i,ii.

- για όλα τα αντικείμενα, υπολογισμός του λόγου κόστος/βάρος, σε χρόνο $O(n)$
- ταξινόμησή τους με φθίνοντα τρόπο, π.χ. με συγχώνευση, σε χρόνο $O(n \log n)$
- αρχικοποιούμε το συνολικό κέρδος σε 0 και το υπολειπόμενο βάρος του σακιδίου (υψ) σε b
- διατρέχουμε την ταξινομημένη λίστα των αντικειμένων και κάθε αντικείμενο επιλέγεται αν τη δεδομένη στιγμή η προσθήκη του στον σάκο δεν ξεπερνά τη χωρητικότητά του, αυξάνοντας κατάλληλα το κόστος και μειώνοντας την υψ, αλλιώς δεν επιλέγεται και προχωράμε στο επόμενο αντικείμενο εφόσον ο σάκος δεν είναι πλήρης, σε χρόνο $O(n)$
- επιστρέφεται το κόστος του σακιδίου

Η πολυπλοκότητα της διαδικασίας είναι $\max\{O(n), O(n \log n)\} = O(n \log n)$.

iii.

Διάταξη [(κόστος, βάρος) $\rightarrow$ λόγος]:

(16,8) $\rightarrow$ 2, (9,5) $\rightarrow$ 1.8, (7,4) $\rightarrow$ 1.75, (15,9) $\rightarrow$ 1.67, (10,6) $\rightarrow$ 1.67, (1,1) $\rightarrow$ 1

υπολειπόμενη χωρητικότητα σακιδίου: $\text{res_cap} = b = 12$

συνολικό κόστος: $\text{total_cost} = 0$

αντικείμενο 1: $\text{res_cap} = 12 - 8 = 4$, $\text{total_cost} = 0 + 16 = 16$

αντικείμενο 2: δεν μπορεί να επιλεγεί γιατί $\text{res_cap} = 4 < 5$

αντικείμενο 3: $\text{res_cap} = 4 - 4 = 0$, $\text{total_cost} = 16 + 7 = 23$

τερματισμός αφού $\text{res_cap} = 0$.

επιστροφή total_cost .

iv. Το πρόβλημα απόφασης 0-1 του σακιδίου είναι NP-πλήρες, ανάγεται σε αυτό το NP-πλήρες πρόβλημα του αθροίσματος υποσυνόλου και μία λύση που δίνεται (επιλεγμένα αντικείμενα) μπορεί να ελεγχθεί σε γραμμικό χρόνο αν πληροί τον περιορισμό βάρους του σακιδίου και έχει συνολικό κόστος σύμφωνο με τον στόχο, άρα δεν έχει γνωστό πολυωνυμικό αλγόριθμο επίλυσης.

v. Ο αριθμός υποπροβλημάτων είναι αντίστοιχος των θέσεων του διδιάστατου πίνακα που δημιουργείται και έχει $(n+1)$ γραμμές $(\# \text{αντικειμένων} + 1)$ και $(b+1)$ στήλες (μονάδες βάρους χωρητικότητας σακιδίου + 1), άρα είναι $(n+1) \cdot (b+1)$.

Συνέχεια ν.

Η αναδρομική σχέση βρίσκεται στο κείμενο με μπλε γράμματα του σχετικού αλγορίθμου δυναμικού προγραμματισμού, με αρχικοποίηση των τιμών στις 2 γραμμές πριν την 1^η for:

- $W=b, v_i = c_i, w_i = a_i$

Algorithm 1: Dynamic Programming Algorithm for 0-1 Knapsack Problem

Data: $W, v_1, v_2, v_3, \dots, v_n, w_1, w_2, w_3, \dots, w_n$

Result: $M[n, W]$

$M[0, w] \leftarrow 0, \forall w \text{ 0 to } W$

$M[i, 0] \leftarrow 0, \forall i \text{ 0 to } n$

for $i \leftarrow 1$ **to** n **do**

for $w \leftarrow 1$ **to** W **do**

if $w_i \leq w$ **then**

 | $M[i, w] = \max(M[i-1, w-w_i] + v_i, M[i-1, w])$

else

 | $M[i, w] = M[i-1, w]$

end

end

end

return $M[n, W]$

ΤΕΛΕΥΤΑΙΟ ΦΡΟΝΤΙΣΤΗΡΙΟ...



Πηγές Φ11:

Σημειώσεις / διαλέξεις μαθήματος, προτεινόμενα βιβλία, υλικό μαθήματος προηγούμενων ετών